

Install the staging client and connect

A 9-step setup guide — from a fresh VS Code to a client connected to staging. · ~5 minutes

Tech Support from RocketRide Engineers: [Discord](#)

Same things covered in this youtube Demo Video:

<https://www.youtube.com/watch?v=IFPQmniW8OA>

Before you start

- VS Code is installed and up to date.
- You have a RocketRide account that exists on staging. staging.rocketride.ai
- pnpm is installed (npm install -g pnpm). The app toolchain — dependency install, the watch loop, the vendored platform packages — runs pnpm, never npm.

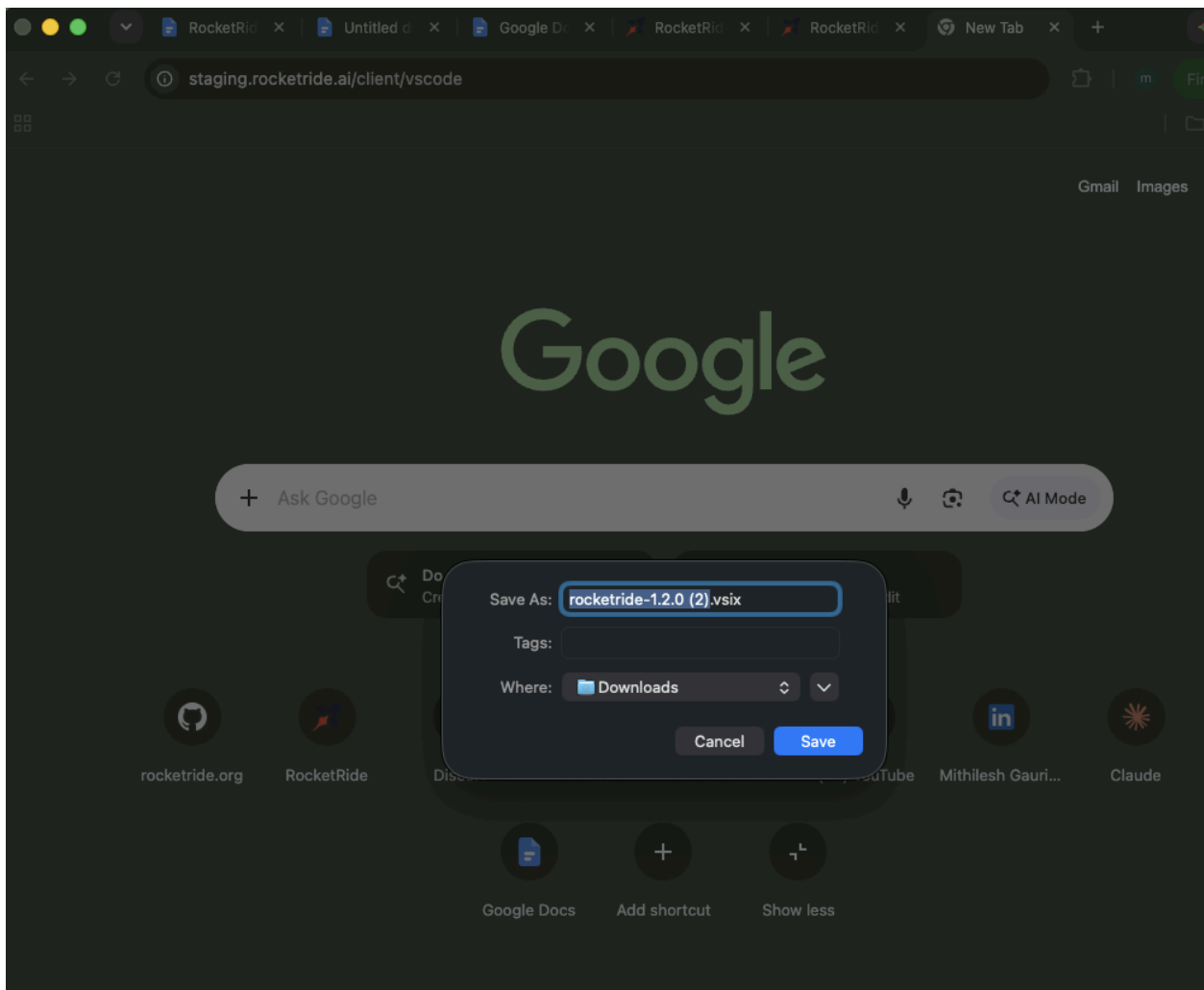
What a coding agent can do for you. A coding agent — Claude Code, or the one built into your editor — can take over everything here that is an API call or a file edit: scaffolding the app, writing its UI and pipeline, validating and running that pipeline, deploying a version, reading a build log, publishing to a rung.

The rest is yours: installing the VSIX and connecting to staging (Steps 1–8), redeeming the credit code (Step 9), registering your developer ID (walkthrough Step 9), and judging whether the running app looks right (Steps 17–18).

Part 1 · Install the staging client

Step 1 Download the staging VSIX in your browser

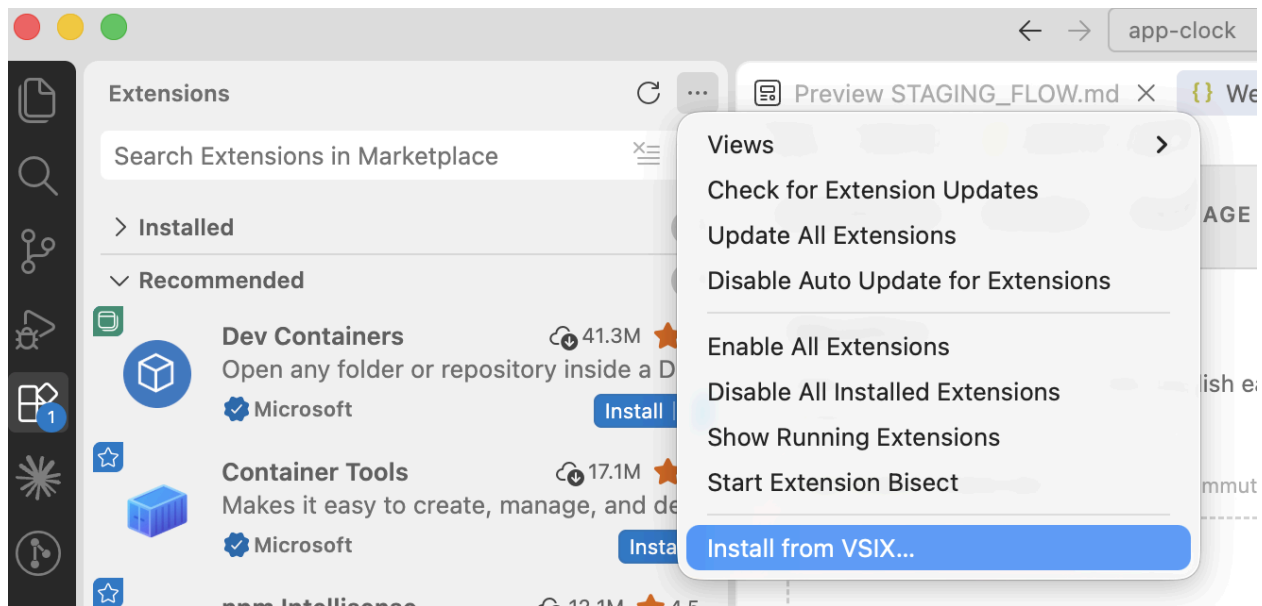
Go to <https://staging.rocketride.ai/client/vscode> in a browser. There is no page to read — the file downloads straight away. At the time of writing you get rocketride-1.2.0.vsix.



Step 2 Open the Extensions overflow menu

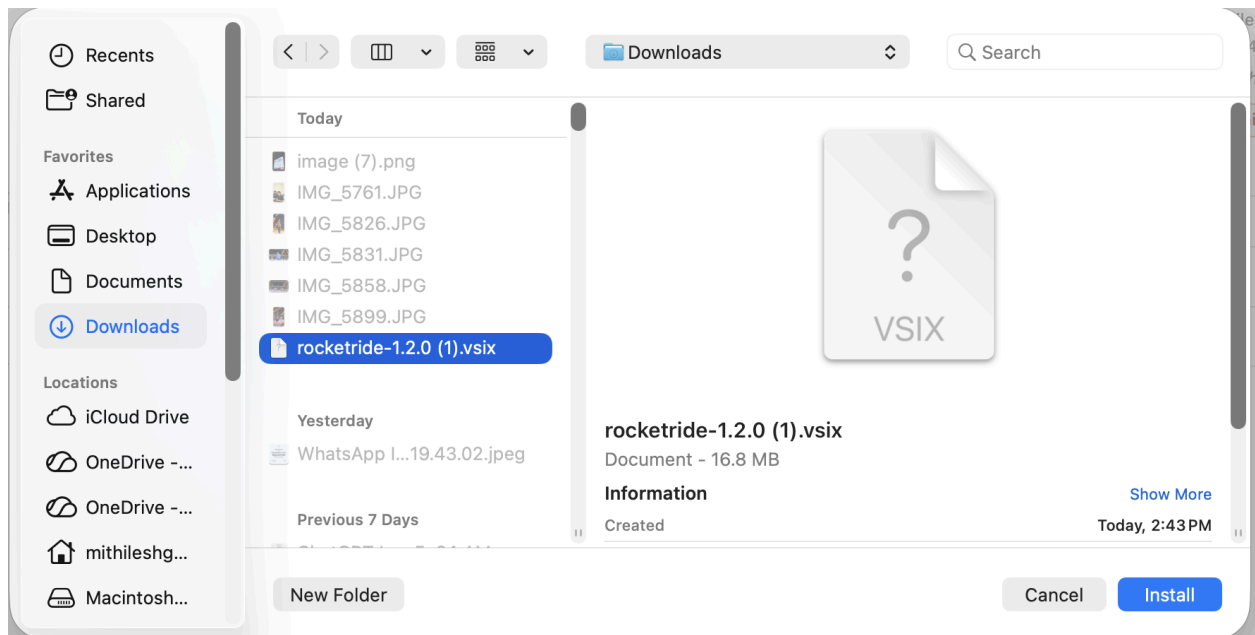
IMPORTANT: Remove the existing Rocketride Extension if you already have one

In VS Code, open the Extensions view (Cmd/Ctrl + Shift + X) and click the three dots in the upper right of the panel.



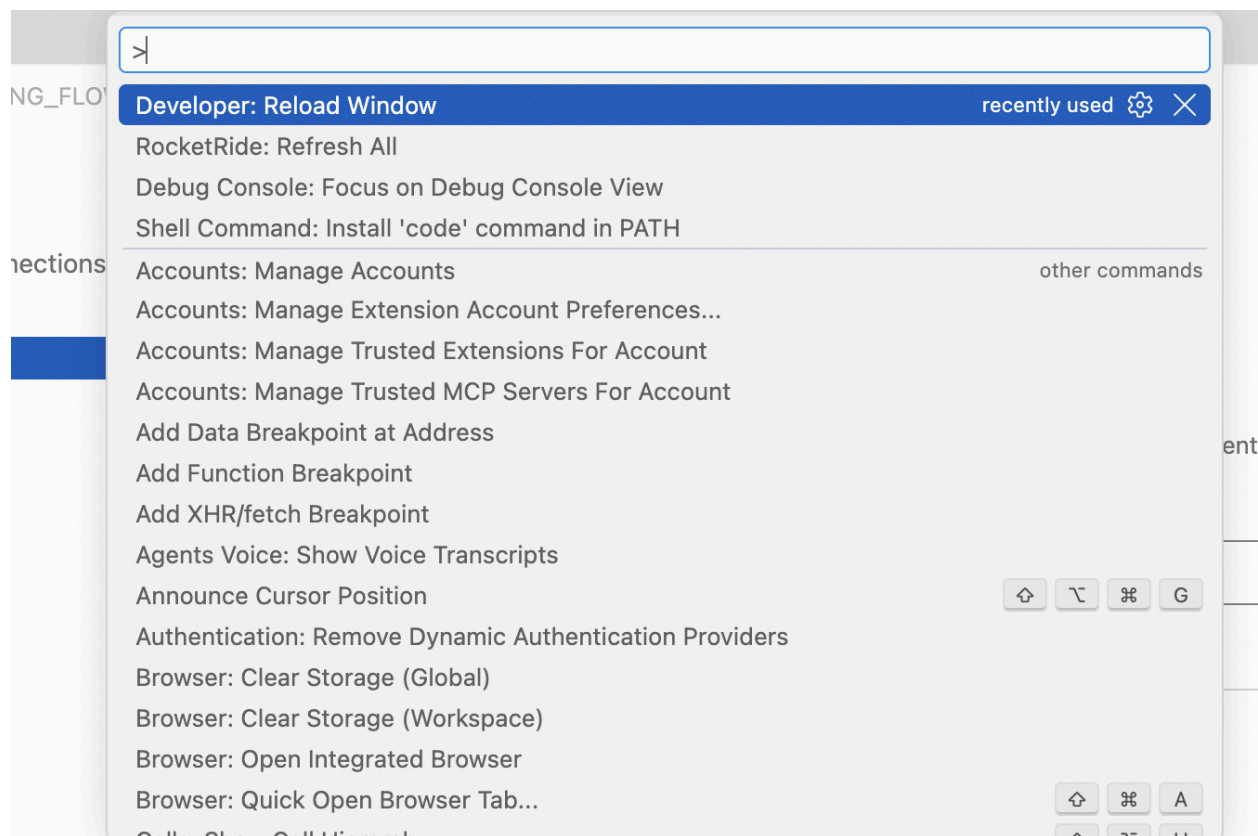
Step 3 Choose Install from VSIX and pick the file

From the overflow menu, choose Install from VSIX... and pick the file you downloaded in Step 1 (rocketride-1.2.0.vsix, usually in your Downloads folder). VS Code installs the extension and then offers a reload prompt.



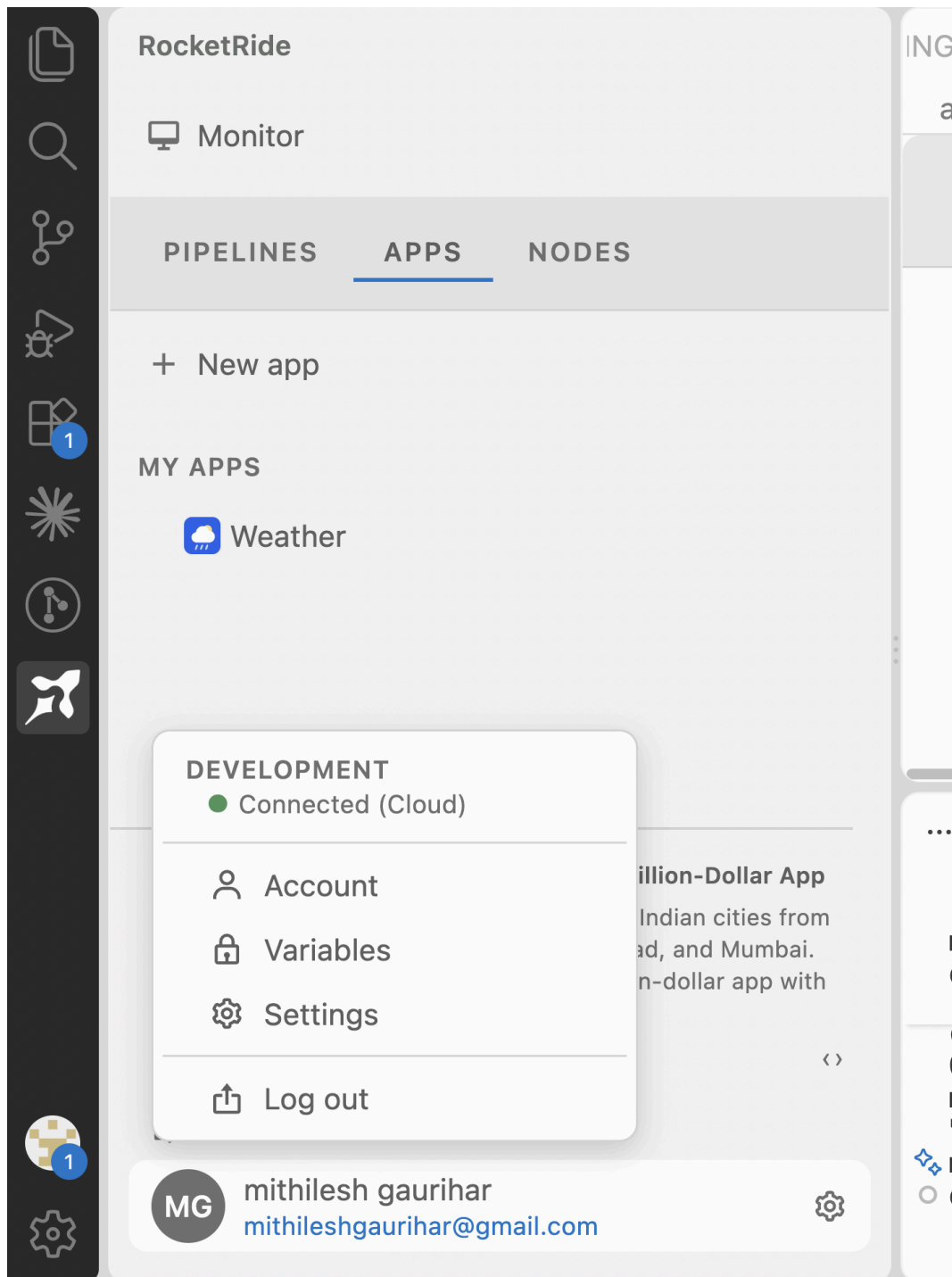
Step 4 Reload VS Code

In VS Code, open the Command Palette view (Cmd/Ctrl + Shift + P) and Enter: Reload Window and press enter



Part 2 · Point Your Client to Staging

Step 5 Open connection settings and select Cloud



Step 6 Click Use custom server

The default cloud endpoint is production. Click **Use custom server** to point the client somewhere else.

Enter <https://staging.rocketride.ai> and confirm.

Preview STAGING_FLOW.md X Weather X RocketRide Settings X Weather app with RocketR... X

Settings

Configure connections, pipelines, and integrations for the RocketRide extension.

Development

Deployment

Pipeline

Integrations

Development

Where pipelines run during development. Cloud and Direct Connect modes require authentication.

Connection mode

RocketRide Cloud

Choose where your server runs for development and deployment

 Sign in with your RocketRide account to connect to the cloud.

☒ Use custom server

Sign-in and all traffic target this server.

☒ mithilesh gaurihar

Sign-in staged. Save settings to apply.

Undo

Cancel Save

Step 7 Sign in when the browser window opens

The sign-in is an OAuth flow owned by the client — you never paste a token or an API key anywhere. Your name and email then appear at the bottom of the Monitor panel.

Step 8 Click Save

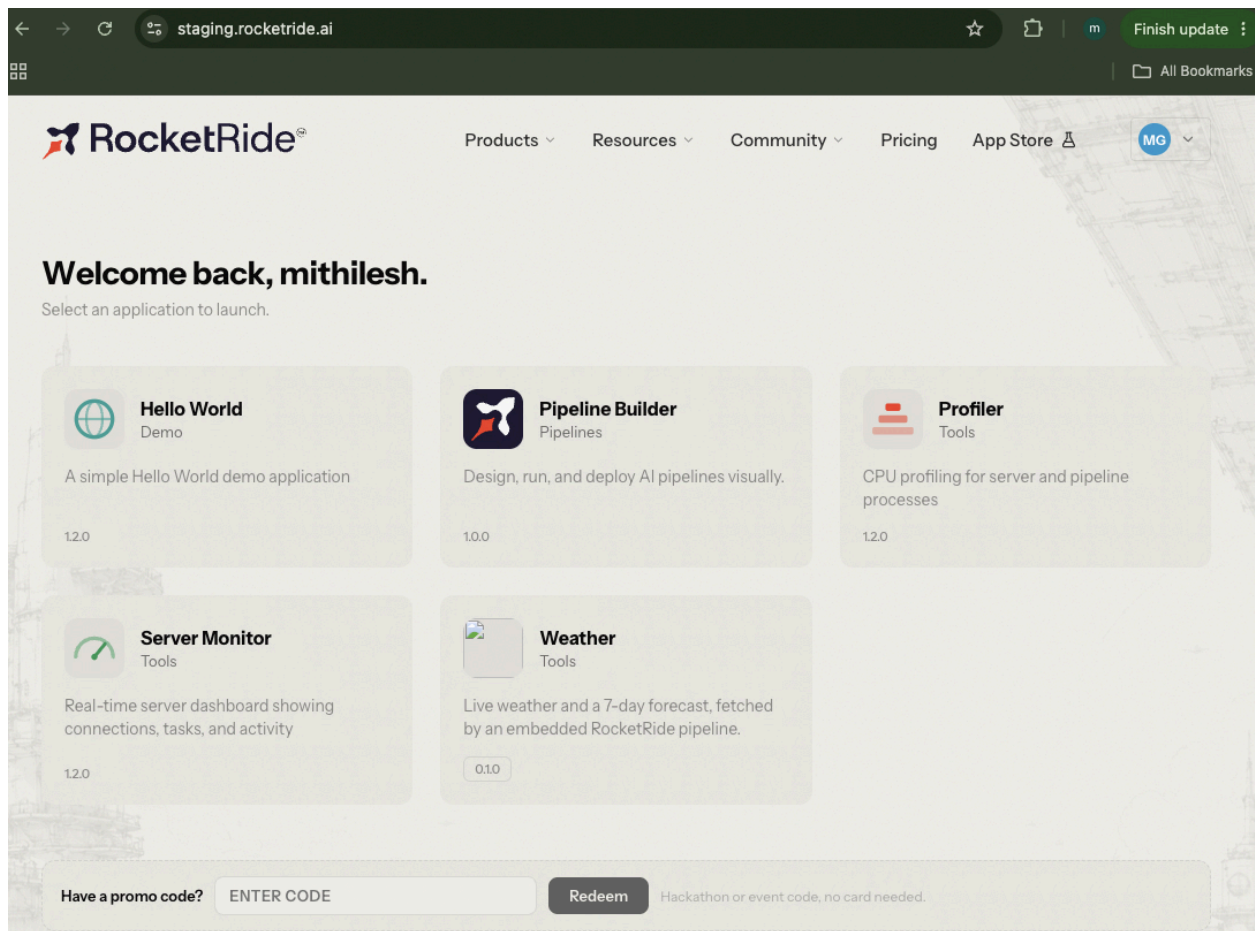
Nothing is applied until you save. If you close the settings panel before saving, the connection reverts to the default cloud endpoint and you have to repeat Steps 5 and 6.

Step 9 Redeem your hackathon credit code

Every pipeline run draws on your organization's token balance, so redeem the hackathon code before you start building. Go to staging.rocketride.ai in a browser and sign in.

The landing page is the app launcher — along the bottom of it is a **Have a promo code?** bar.

Type promo code into ENTER CODE and click Redeem. **No card is needed.**



Set up your staging environment and ship your first app

An 18-step walkthrough of the RocketRide IDE — from an empty workspace to a deployed app. · ~15 minutes

Before you start

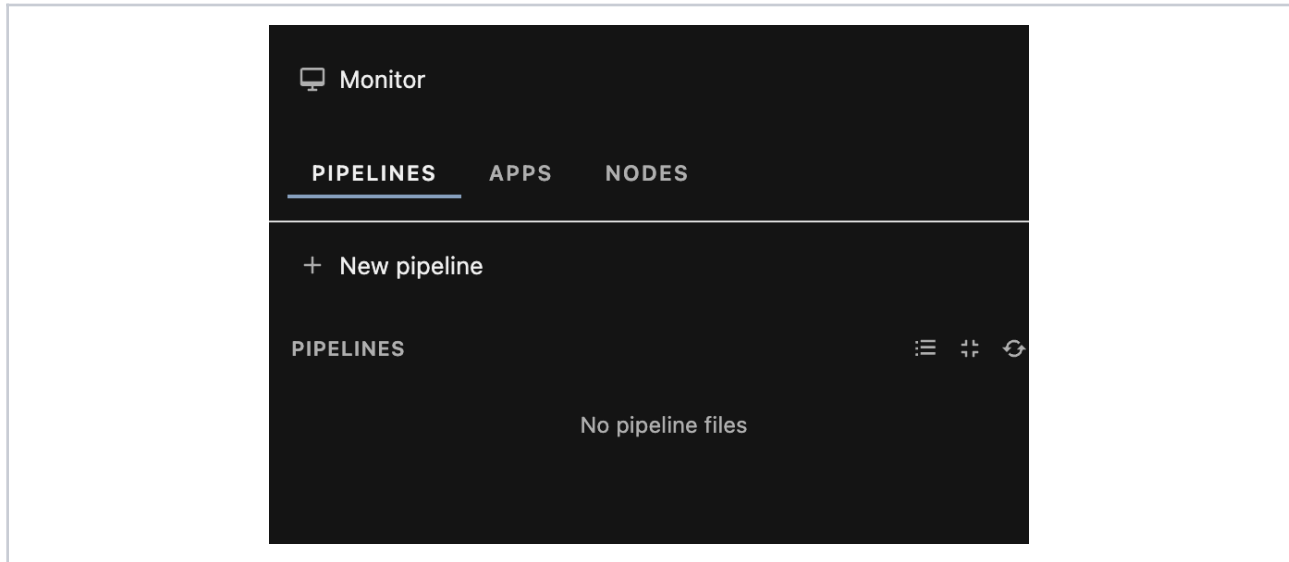
- The RocketRide IDE is open on your hackathon staging workspace.
- You are signed in — your name and email appear at the bottom of the Monitor panel.
- You know your organization's developer ID, or you are the person who will claim it (Part 4).

Read this first: Work through Parts 1–4 with a throwaway app name. Claiming your developer ID changes the namespace every future app is created in, so it is much cleaner to do it once on a scratch app than to rename a real one later.

Part 3 · Find your way around

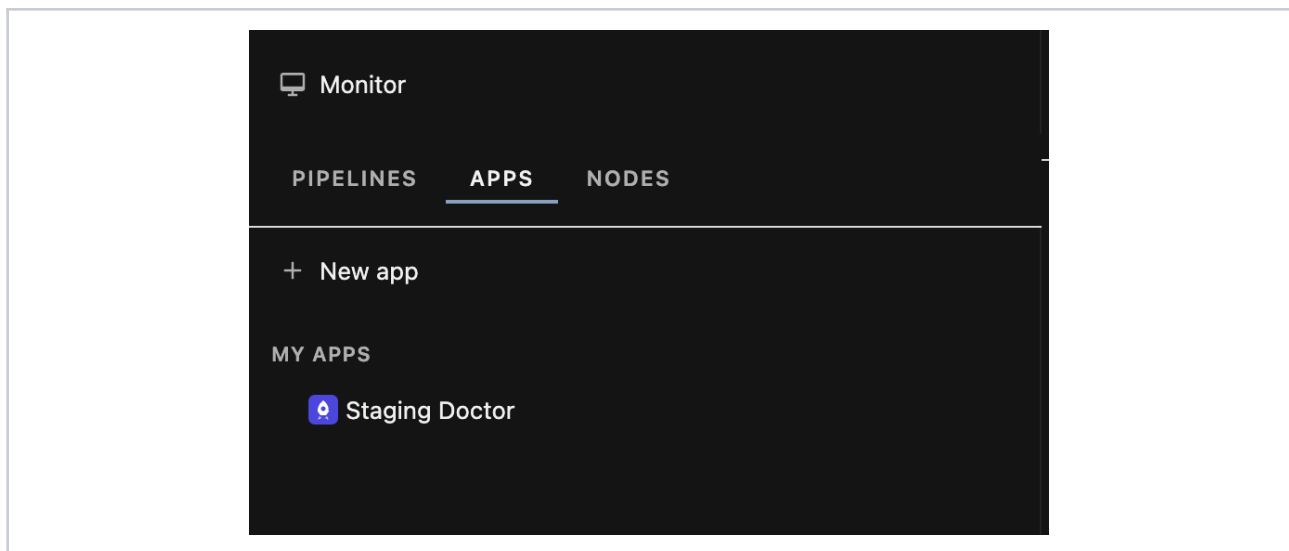
Step 1 Open the Monitor panel

Monitor is the control room for the workspace. It has three tabs — Pipelines, Apps and Nodes. A fresh hackathon workspace shows “No pipeline files”, which is expected; you are building an app, not a pipeline.



Step 2 Switch to the Apps tab

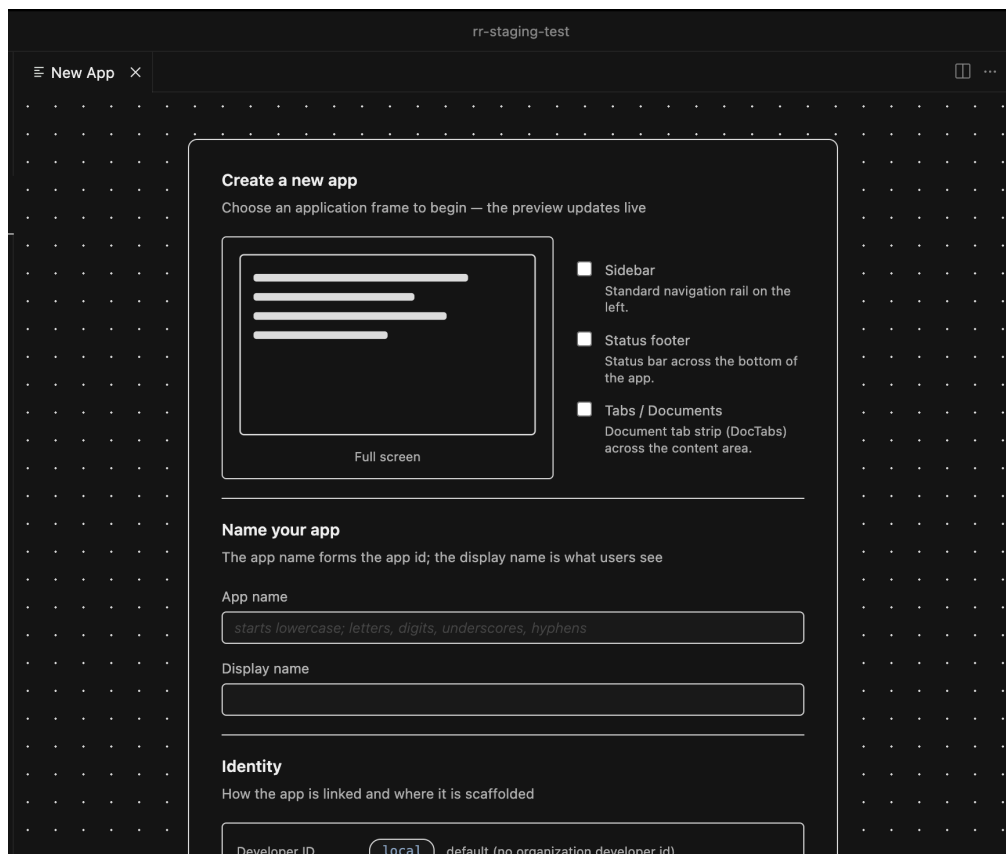
Apps lists everything scaffolded in this workspace under MY APPS, with + New app at the top. This is where you will start and return to for every app you build.



Part 4 • Scaffold a scratch app

Step 3 Click + New app and pick an application frame

The frame decides the shell your app is generated with. Full screen is the default and is the right choice for a first run. Sidebar, Status footer and Tabs / Documents are optional chrome you can add later — leave them unchecked. The preview on the left updates as you toggle them.



Step 4 Name the app and read the Identity block

Scroll down in the same tab. **App name** becomes the app id — it must start lowercase and use only letters, digits, underscores and hyphens. **Display name** is what users see and can be anything.

The **Identity** block underneath is the important part. Here Developer ID reads `local` — that is the fallback used when your organization has not claimed a developer ID yet, so the app is linked as `local.<name>` and scaffolded into `apps/<name>-ui/`.

Create App stays disabled until App name has a value.

Full screen

- ☒ Sidebar
Standard navigation rail on the left.
- ☐ Status footer
Status bar across the bottom of the app.
- ☐ Tabs / Documents
Document tab strip (DocTabs) across the content area.

Name your app

The app name forms the app id; the display name is what users see

App name

starts lowercase, letters, digits, underscores, hyphens

Display name

Identity

How the app is linked and where it is scaffolded

Developer ID **local** default (no organization developer id)

Linkage name **local.<name>**

Location **apps/<name>-ui/**

No organization developer id — using the "local" default.

Cancel Create App

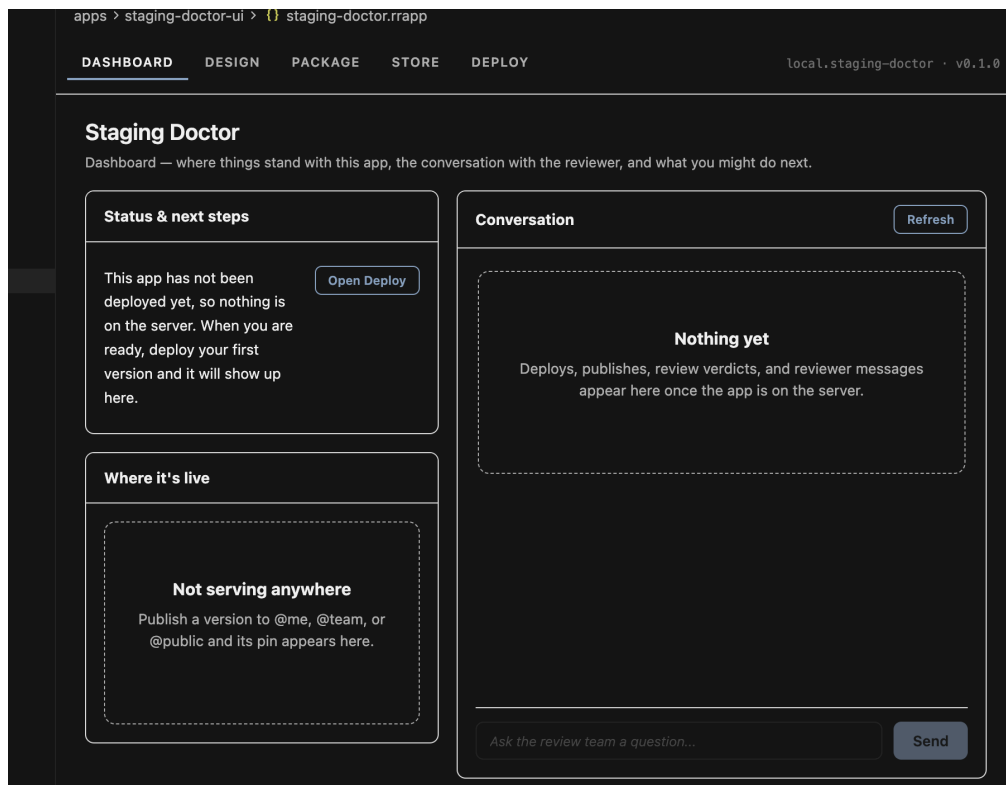
Use a scratch name here. This walkthrough uses staging-doctor. Your real app comes in Part 5, after the namespace is fixed.

Part 5 • The five tabs of an app

Creating the app opens a .rapp tab with five sub-tabs. Here is what each one is for — you only need Package and Deploy to ship.

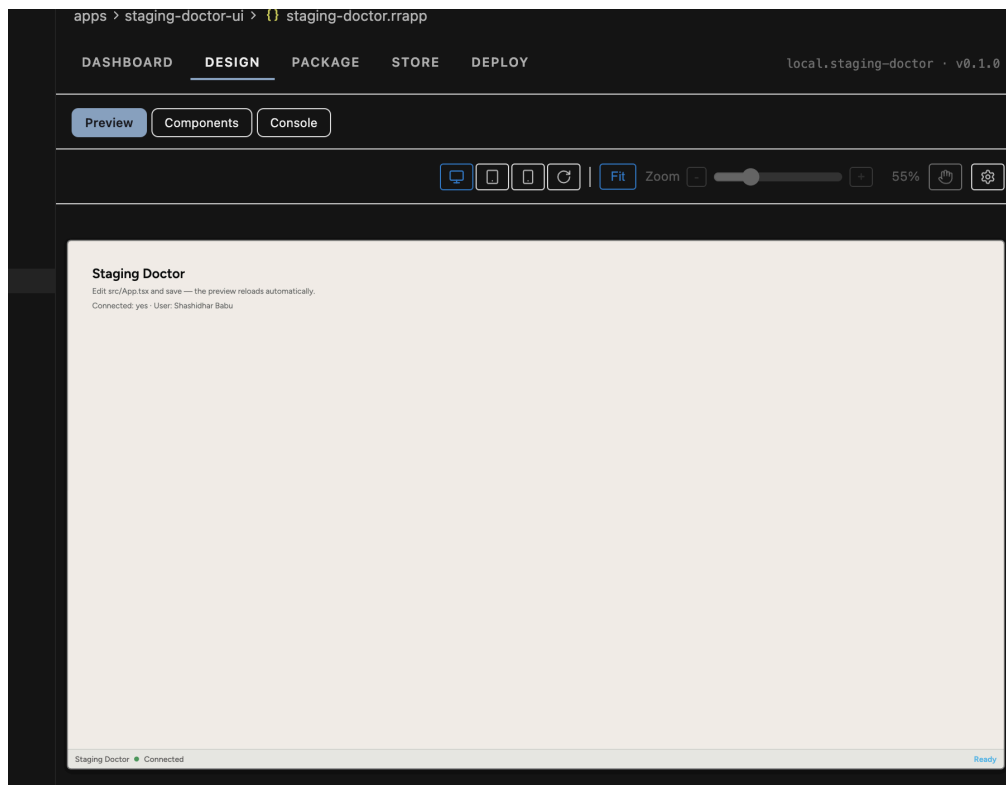
Step 5 Dashboard — where things stand

Status and next steps, the conversation with the review team, and where the app is currently served. Everything is empty until your first deploy, which is correct at this stage.



Step 6 Design — live preview

Edit `src/App.tsx` and save; the preview reloads automatically. Use the device buttons and zoom to check layouts. The green **Connected** marker in the footer means the preview is wired to the running app — if it goes red, your build is broken.



Step 7 Package — what ships with the app

App id, display name, description, icon, README, and any workspace paths your app imports beyond its own folder (for example apps/ shared). The **Readiness** panel on the right must be all green before you deploy — a missing include path fails the server build.

Leave Strict type checking on. It makes the server verify the app with your own tsconfig before building, which catches errors at deploy time instead of at demo time.

apps > staging-doctor-ui > {} staging-doctor.rapp

DASHBOARD DESIGN **PACKAGE** STORE DEPLOY local.staging-doctor · v0.1.0

Staging Doctor

Package — everything this app needs to build, deploy, and publish personally: identity, icon, README, and the workspace paths it ships with. Store-only concerns (pricing, review) live on the Store tab.

Identity

APP ID

DISPLAY NAME

DESCRIPTION

STRICT TYPE CHECKING

☒ The server verifies the app with your own tsconfig before building — turning this off deploys even with type errors.

Assets

ICON (SVG)

Readiness

Everything is in place — deploy it and publish to your desktop or a team whenever you like.

- ✓ App id namespaced local.staging-doctor
- ✓ Display name Staging Doctor
- ✓ Icon ./icon.svg
- ✓ README ./README.md

Include paths

Workspace directories or files your app imports beyond its own folder (for example `apps/shared`). They are packed into every deploy and installed by the server build — a missing one fails the build.

workspace-relative

Step 8 Store — only if you are going public

Pricing mode and the requirements every public version must pass. You can skip this entirely for the hackathon: @me and @team deploys do not touch the store.

apps > staging-doctor-ui > {} staging-doctor.rapp

DASHBOARD DESIGN PACKAGE **STORE** DEPLOY local.staging-doctor · v0.1.0

Staging Doctor

Store — the commerce posture and the public rung: pricing mode, plans, and the requirements every public version must pass. Only needed to distribute on the App Store; @me and @team deploys skip all of this. Every public version is reviewed — submission happens on the Deploy tab.

Store listing

● local

APP ID

MODE

Store requirements

- ✓ Description

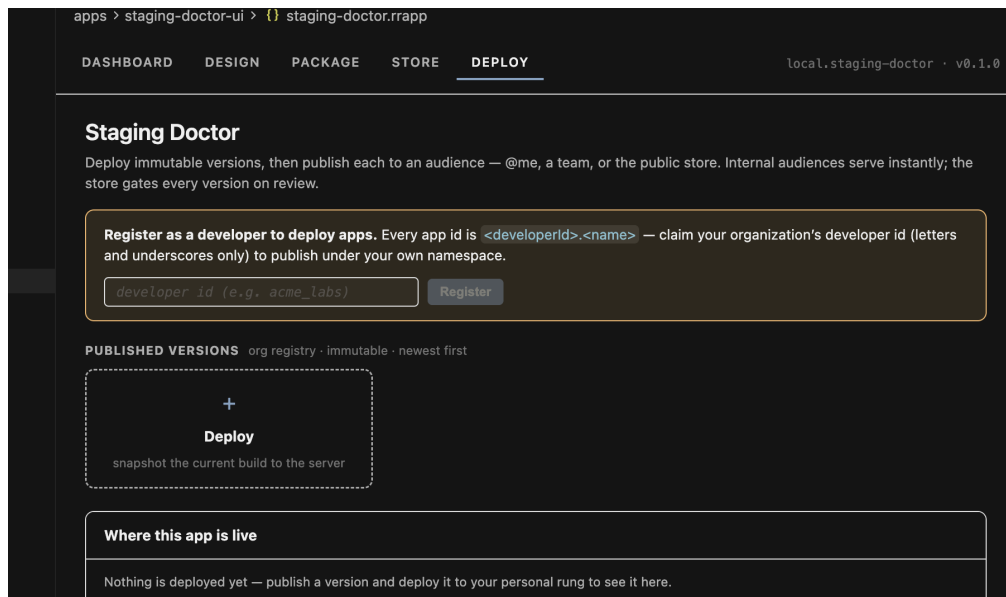
everything the store requires is in place — deploy a version and submit it for review on the Deploy tab.

Part 6 · Claim your developer ID

This is the step that turns a local scratch workspace into a real staging environment. Do it once.

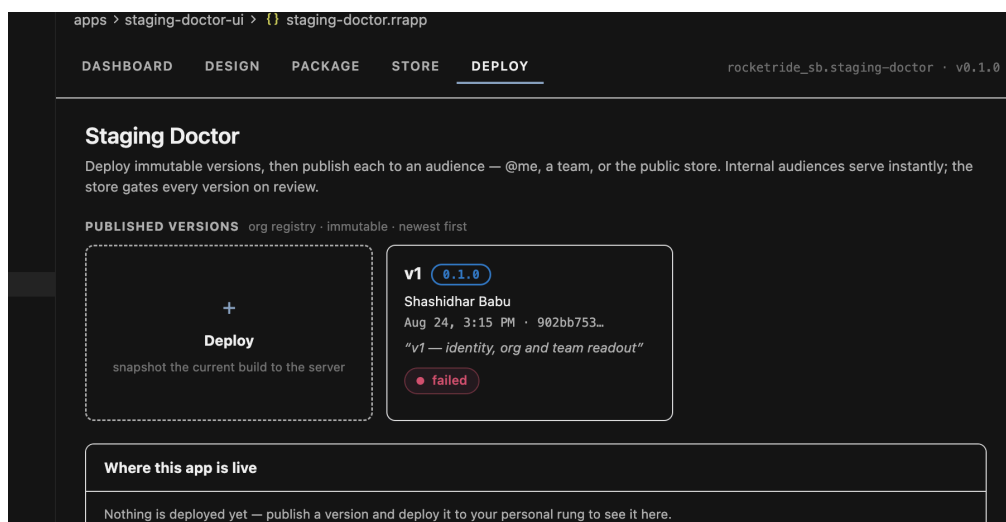
Step 9 Open the Deploy tab and register

Every app id has the form `<developerId>.<name>`. Until a developer ID exists, the amber banner blocks deployment. Enter your organization's ID — **letters and underscores only** — and click Register. This walkthrough uses `rocketride_sb`.



Step 10 Deploy your first version

The banner disappears and the header now reads `rocketride_sb.staging-doctor`. Click **+ Deploy** to snapshot the current build to the server. Each deploy produces an immutable version card with a number, a semantic version, your name, a timestamp, a content hash and your deploy message — you cannot overwrite a version, only publish a newer one.



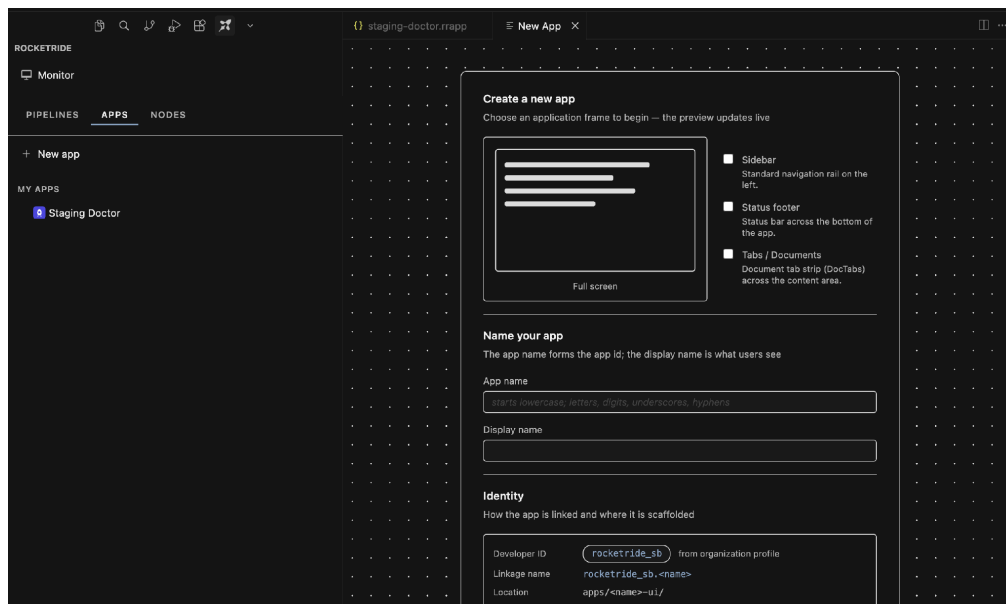
Part 7 · Build your own app

Let a coding agent build it. Describe the problem you want solved — the use case, not the implementation — and let the agent scaffold the app, write the UI and the pipeline, and iterate with you. Point it at `.rocketride/docs/`, `services-catalog.json` and `schema/` before it writes a line: those are ground truth for component names, config fields and SDK signatures, and an agent that guesses writes code that looks right and fails on the server.

The Weather app on the staging launcher came from a one-line brief this way — a `.pipe` that geocodes a place and pulls a 7-day forecast, a UI on stock shell components, and a check script that runs one real lookup. Ask for that check script, and have the app parse model output defensively: an LLM answer is data, not a contract.

Step 12 Go back to Monitor → Apps → + New app

Your scratch app is now listed under MY APPS. Start a new one for the app you actually want to demo.



Step 13 Confirm Identity now reads from your organization

Compare this with Step 4. Developer ID is now `rocketride_sb` with the label “from organization profile”, and Linkage name is `rocketride_sb.<name>`. Every app you create from here lands in the right namespace automatically.

{} staging-doctor.rapp

New App

Full screen

- ☒ Sidebar
Standard navigation rail on the left.
- ☒ Status footer
Status bar across the bottom of the app.
- ☐ Tabs / Documents
Document tab strip (DocTabs) across the content area.

Name your app
The app name forms the app id; the display name is what users see

App name
starts lowercase; letters, digits, underscores, hyphens

Display name

Identity
How the app is linked and where it is scaffolded

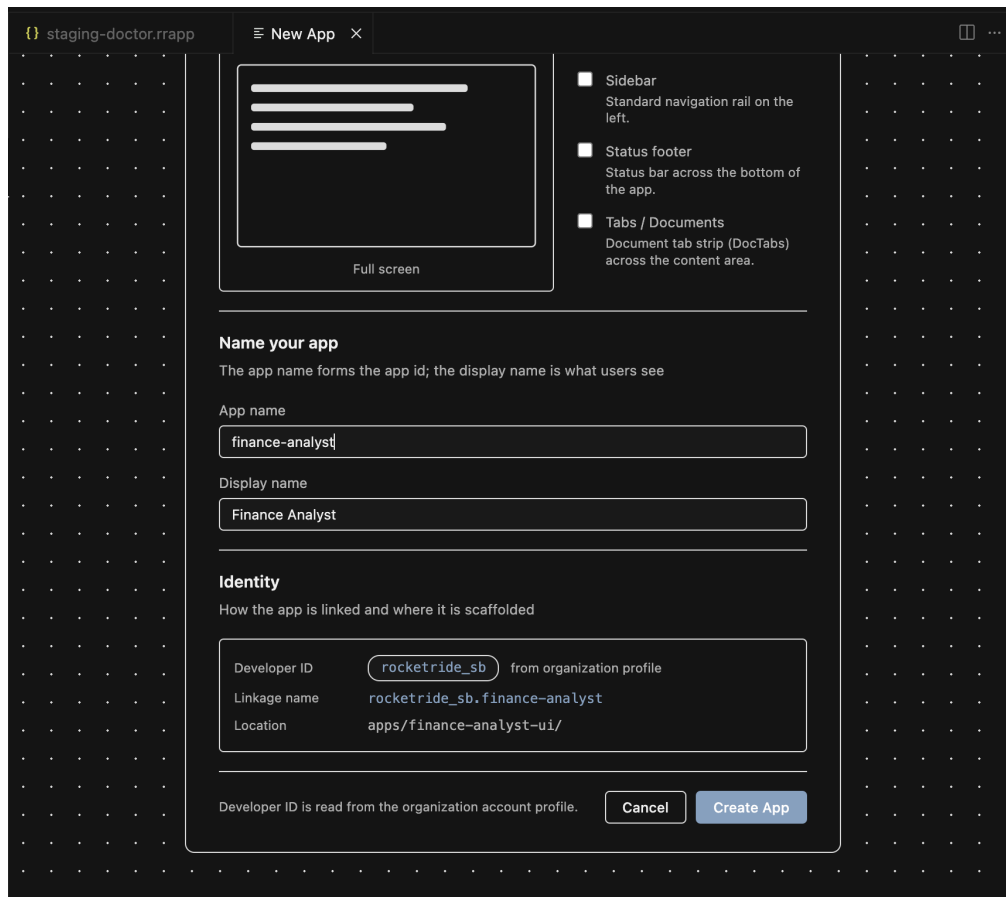
Developer ID	rocketride_sb	from organization profile
Linkage name	rocketride_sb.<name>	
Location	apps/<name>-ui/	

Developer ID is read from the organization account profile.

Cancel Create App

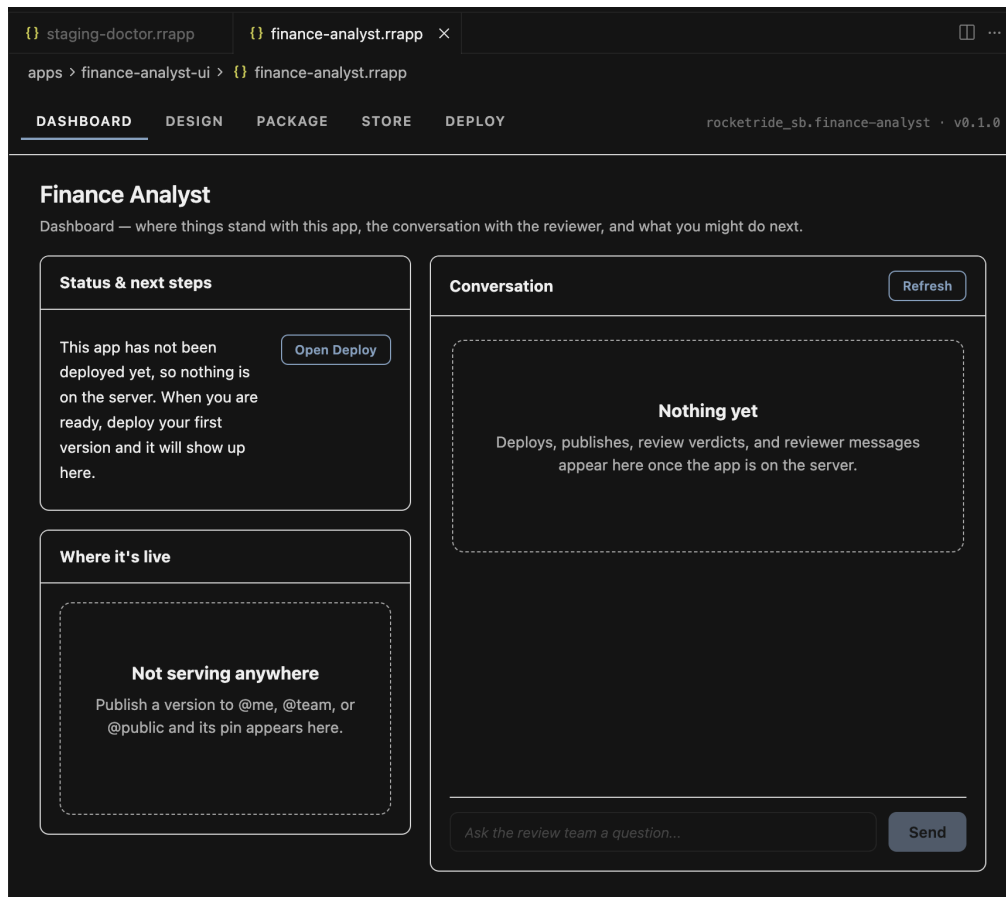
Step 14 Name it and click Create App

Fill in App name and Display name. Watch the Identity block resolve live — here `finance-analyst` gives linkage name `rocketride_sb.finance-analyst` at `apps/finance-analyst-ui/`. Create App is now enabled.



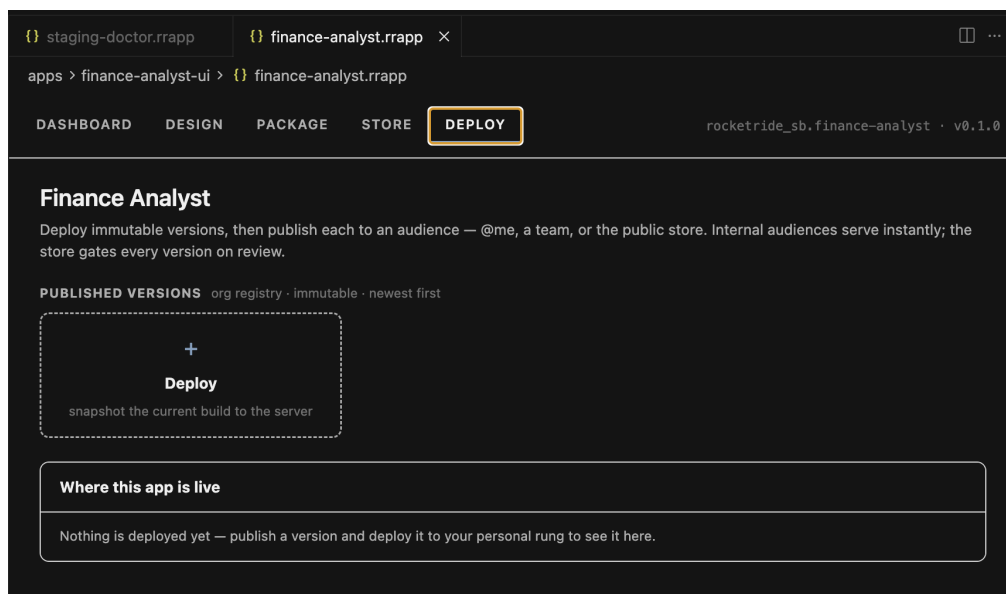
Step 15 Your app opens on its Dashboard

A new `.rapp` tab opens beside the scratch one, with the same five sub-tabs and the correct app id in the header. Build in Design, fill in Package, and come back here to check status.



Step 16 Deploy it

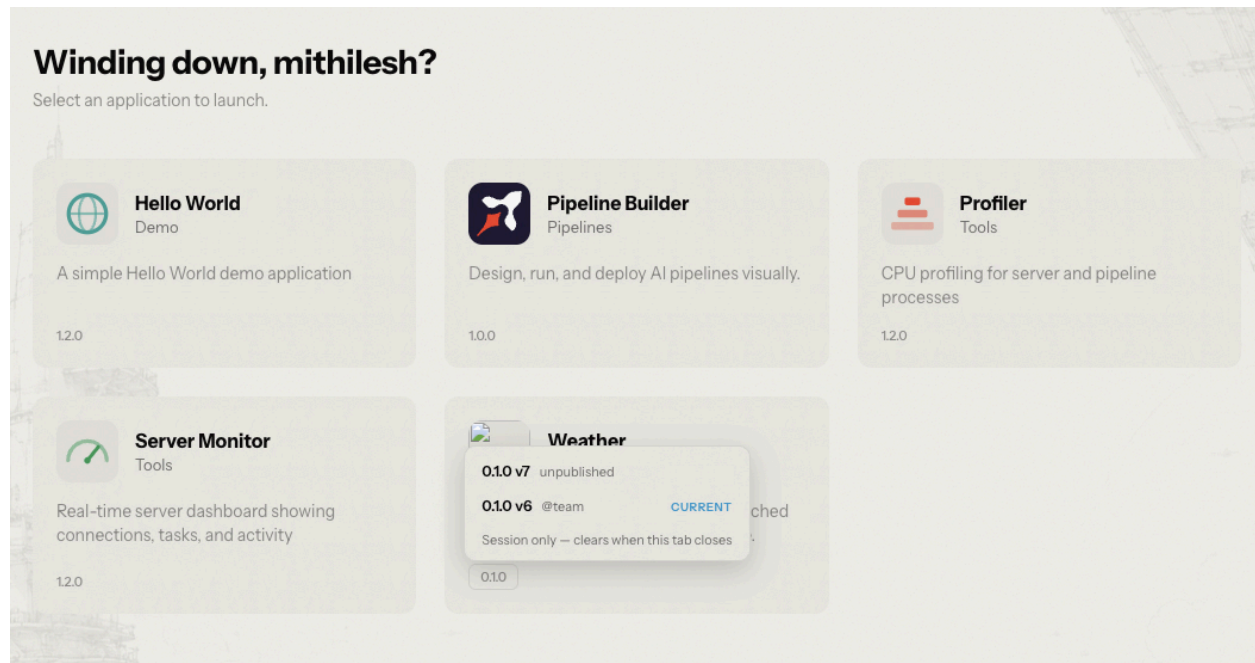
Deploy tab → **+ Deploy** → snapshot. No registration banner this time. Publish the version to @me to test it yourself, or @team to share it with the rest of your hackathon table — both serve instantly. @public goes through review and is not needed for the hackathon.

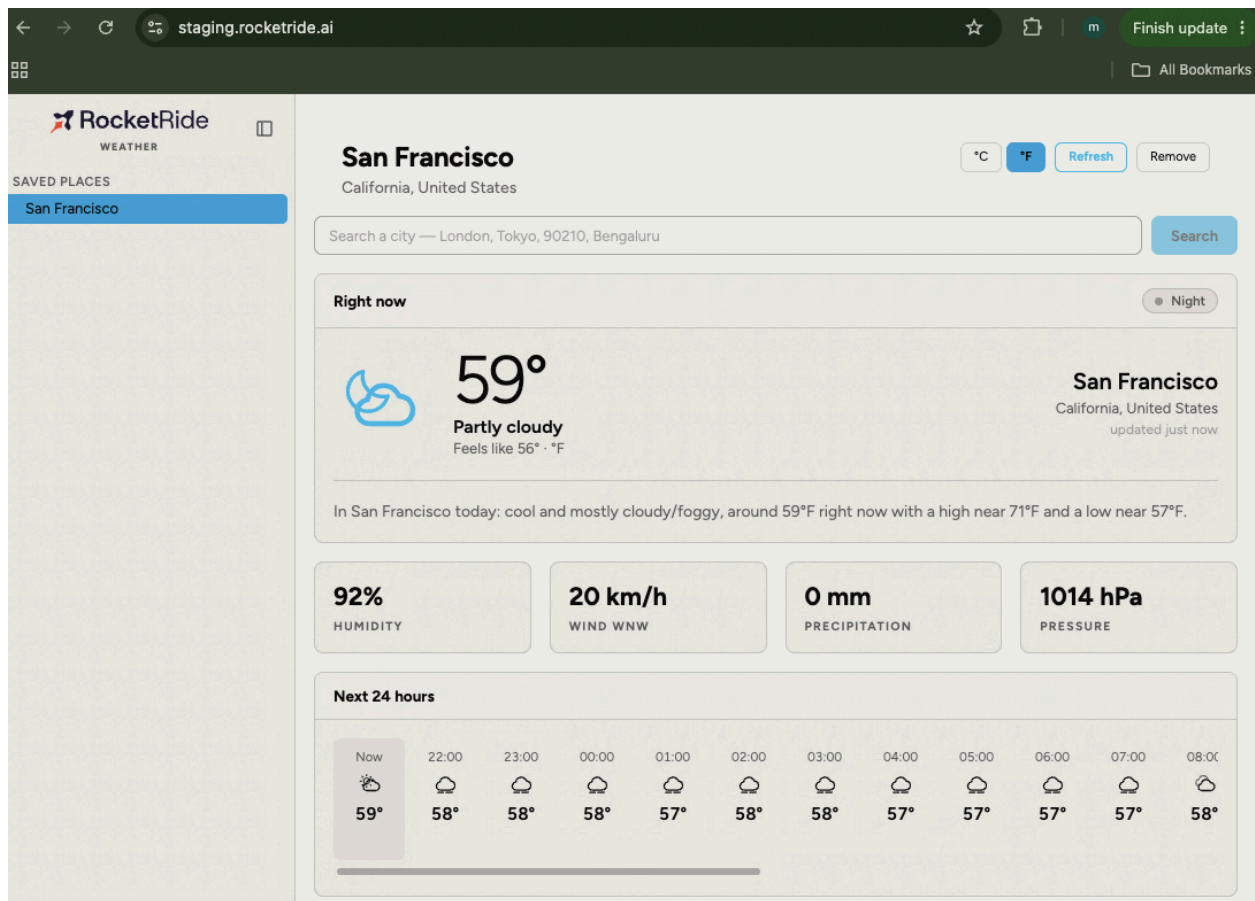


Step 17 See your app running on staging

Open <https://staging.rocketride.ai> in a browser and sign in with the same account. A version published to @me appears on your own desktop; one published to @team appears for everyone on that team. Click the tile to launch it — this is the deployed build, served from its immutable version directory, not your local dev server. To open a specific version, use the version drop list on the app tile or add ?appid=<your.app>&version=<number> to the URL.

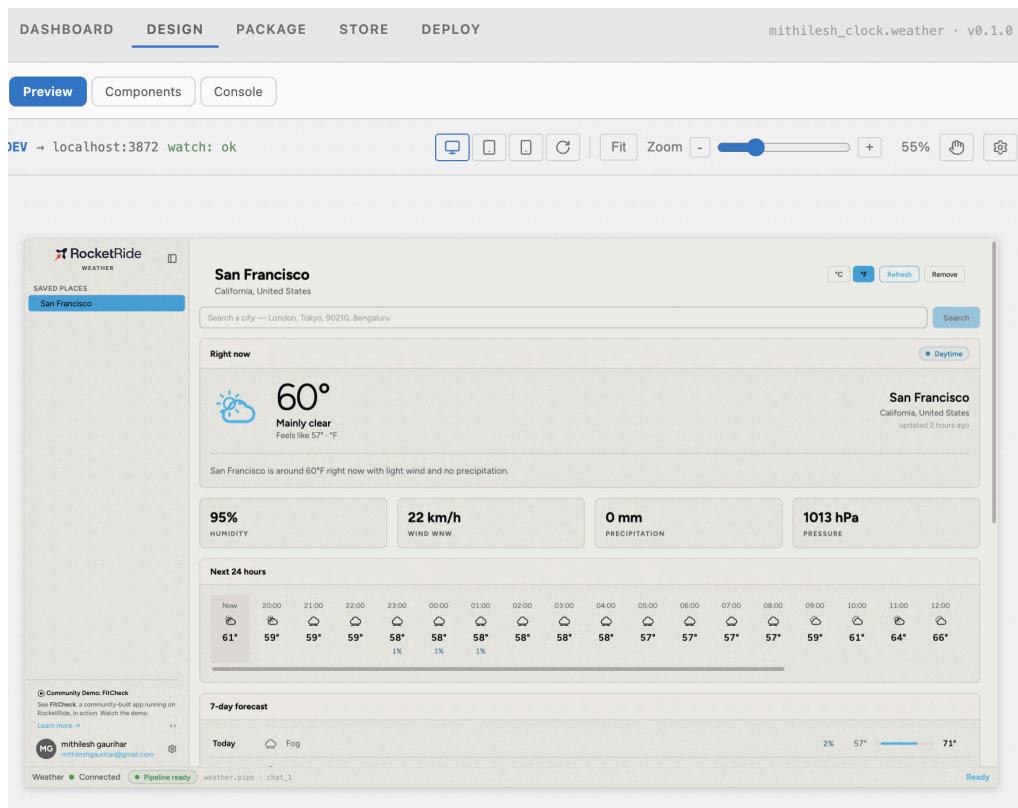
One catch: while your watch is still running in VS Code, the dev overlay takes precedence and you will see your local build here instead of the deployed one. Close the App Builder panel to stop the watch before you verify what a published version actually serves.





Step 18 Compare it with the Design tab preview

Back in VS Code, Design → Preview runs the same shell against your working copy, rebuilding and reloading on every save. That is the loop you build in; Step 17 is how you confirm what you actually shipped. If the two look different, you are viewing an older published version — deploy again and publish the new one.



Troubleshooting

Symptom	What to do
“Create App” stays greyed out	The App name field is empty or invalid. It must start lowercase and use only letters, digits, underscores and hyphens.
“Register” stays greyed out	Developer IDs accept letters and underscores only — no hyphens, digits or spaces. rocketride_sb works; rocket-ride-1 does not.
A new app still shows Developer ID = local	The New App tab read the profile before you registered. Close the New App tab and open a fresh one.
Deploy card shows failed	Open the card to read the build log. The failing phase is named at the top of the block and the reason is on the last line.
Readiness panel is not all green	Fill the missing item on the Package tab (app id, display name, icon, README). A missing include path fails the server build.
Anything reports “No authorization provided”	Your workspace credentials were cleared — ROCKETRIDE_APIKEY in .env is empty. Run pnpm exec rocketride login in the workspace root; it opens the browser, mints a key and rewrites .env.

Symptom	What to do
You sign in, but the client is signed out again minutes later	Same cause: .env is rewritten with empty key lines while the URIs survive. Run <code>pnpm exec rocketride login</code> again, and report it if it keeps recurring — you cannot deploy or run pipelines in that state.
The app preview shows a full-screen “Sign in required” wall	That is the shell’s own gate, from "authenticated": true in appManifest. Its sign-in opens a separate window that may not complete inside the VS Code webview. Set "authenticated": false in the app’s package.json and let the app handle the signed-out state itself.
Deploy tab is read-only: the app id is outside your developer namespace	The app was scaffolded before you claimed the developer ID, so its id is still local.<name>. Rename it in two places — appManifest.id in package.json and the id field in src/AppDescriptor.ts. They must match or the app will not load.

Quick checklist

- Monitor → Apps → + New app
- Pick Full screen · name the app · Create App
- Deploy tab → register your developer ID (letters and underscores only)
- Package tab → Readiness all green
- Design tab → build, preview shows Connected
- Deploy tab → + Deploy → publish to @me or @team
- staging.rocketride.ai → sign in → launch your published app